SERVO MOTION & AUTOMATION

Curiosity Kickoff

- Can a servo rotate fully like a fan?
- What angles can it move?
- Does it stop at a position?

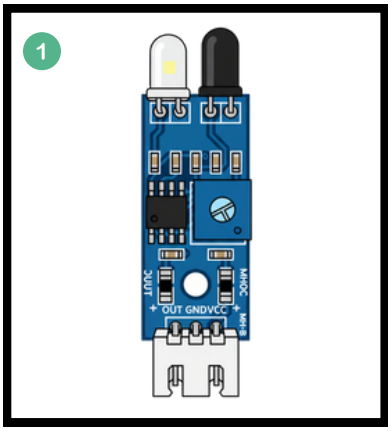
Concept Connection

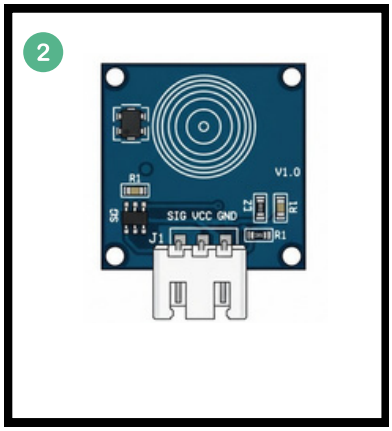
A Servo Motion & Automation system uses a servo motor to control precise movement and positioning. It operates based on input signals, allowing accurate angle rotation. This system is widely used in robotics, automated devices, and control systems where controlled motion and positioning are required.

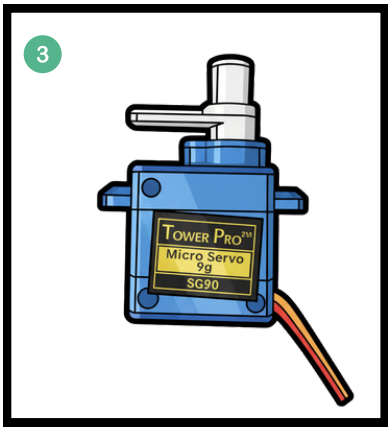
1.IR (Infrared) Sensor :
Detects infrared light, which is invisible to human eyes.Often used for obstacle detection or remote control.
Pinout: 3 pins - VCC: Power, GND: Ground & OUT: Digital signal

2.Touch Sensor
Works like a tiny switch without pressing. Inside, it senses the small change in capacitance when your finger touches it (your body conducts a little electricity). It sends this signal to Arduino as HIGH or LOW.

3.Servo motor
A Servo Motor is a device used for precise control of position, angle, or movement. It works using a control signal (usually PWM) to rotate to a specific angle. Servo motors are commonly used in robotics, automation systems, and electronic projects requiring accurate motion control.







Beginner: Servo Motion

Class Explanation

A Servo Motion system uses a servo motor to achieve precise and controlled movement. It operates based on input signals, allowing accurate rotation to specific angles, making it useful in robotics, automation, and applications requiring controlled positioning and smooth motion.

Materials Required

- Arduino with attached shield
- 1 × Servo
- RMC wires
- USB cable

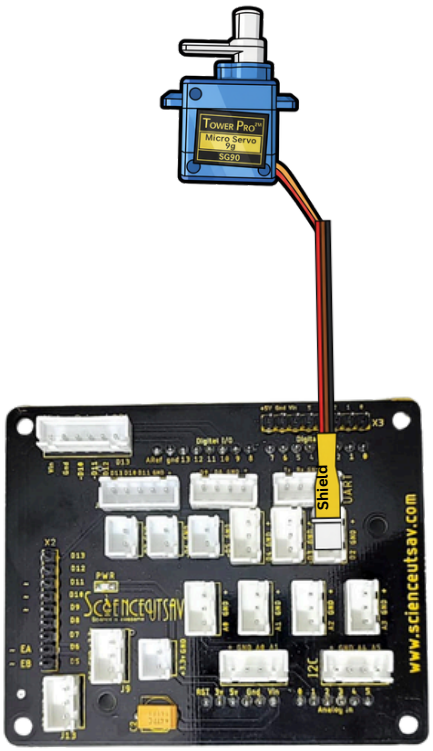
How to Connect

Use RMC wires to connect :

- 1 × Servo→ Arduino pin D2 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <Servo.h>
#define SERVO_PIN 2  // Servo signal connected to pin 2

Servo myServo;
void setup() {
 myServo.attach(SERVO_PIN);
}

void loop() {
 for (int angle = 0; angle <= 180; angle++) {
  myServo.write(angle);
  delay(15);
 }

 for (int angle = 180; angle >= 0; angle--) {
  myServo.write(angle);
  delay(15);
 }
}
```

Code Explanation

- In this Arduino program, a servo motor is connected to pin 2 (SERVO_PIN). The Servo library is used to control the servo.
- The setup() function runs once when the Arduino starts. It attaches the servo to the specified pin using attach().
- The loop() function runs continuously. First, a for loop moves the servo from 0° to 180° step by step using write(), with a small delay for smooth motion.
- Then, another for loop moves the servo back from 180° to 0° in the same way.

So overall, the servo motor continuously sweeps back and forth between 0° and 180°.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Servo Motion Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <Servo.h>

#define SERVO_PIN 2   // Servo on D2

Servo myServo;

void setup() {
  myServo.attach(SERVO_PIN);
}

void loop() {
  // Move from 0° to 90°
  for (int angle = 0; angle <= 90; angle++) {
    myServo.write(angle);
    delay(15);
  }

  // Move back from 90° to 0°
  for (int angle = 90; angle >= 0; angle--) {
    myServo.write(angle);
    delay(15);
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Servo Motion Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Servo Motion Starter

Class Explanation

A Servo Motion Starter is a basic system that introduces how a servo motor works and how to control its movement. It uses simple input signals to rotate the servo to specific angles, helping beginners understand motion control, positioning, and basic automation concepts in electronics.

Materials Required

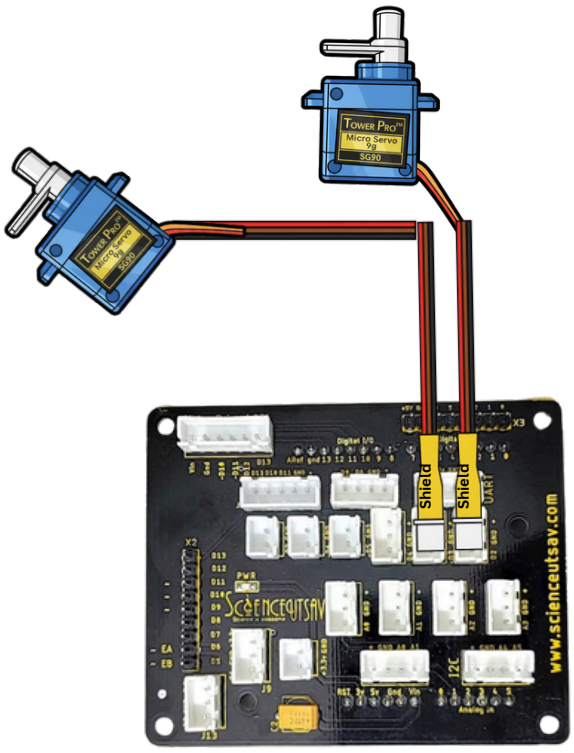
- Arduino with attached shield
- 2 × Servo motor
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × Servo motor → Arduino pin D2 and GND
- 1 × Servo motor → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <Servo.h>

#define SERVO1_PIN 2   // First servo on D2
#define SERVO2_PIN 3   // Second servo on D3

Servo servo1;
Servo servo2;

void setup() {
  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);
}

void loop() {
  // 0 → 180
  for (int angle = 0; angle <= 180; angle++) {
    servo1.write(angle);
    servo2.write(angle);
    delay(15);
  }

  // 180 → 0
  for (int angle = 180; angle >= 0; angle--) {
    servo1.write(angle);
    servo2.write(angle);
    delay(15);
  }
}
```

Code Explanation

- In this Arduino program, two servo motors are connected to pins 2 (SERVO1_PIN) and 3 (SERVO2_PIN). The Servo library is used to control both servos.
- The setup() function runs once when the Arduino starts. It attaches both servos to their respective pins using attach().
- The loop() function runs continuously. First, a for loop moves both servos together from 0° to 180° step by step using write(), with a small delay for smooth motion.
- Then, another for loop moves both servos back from 180° to 0° in the same way.

So overall, both servo motors move together, sweeping back and forth between 0° and 180°.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Servo Motion Starter with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <Servo.h>

#define SERVO1_PIN 2
#define SERVO2_PIN 3

Servo servo1;
Servo servo2;

void setup() {
  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);
}

void loop() {
  // 0 → 90
  for (int angle = 0; angle <= 90; angle++) {
    servo1.write(angle);
    servo2.write(angle);
    delay(15);
  }

  // 90 → 0
  for (int angle = 90; angle >= 0; angle--) {
    servo1.write(angle);
    servo2.write(angle);
    delay(15);
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Servo Motion Starter with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Servo Angle Control

Class Explanation

A Servo Angle Control system uses control signals, typically PWM (Pulse Width Modulation), to set a servo motor to a specific angle. It allows precise positioning and movement, making it useful in robotics, automation, and applications requiring accurate angle control and smooth operation.

Materials Required

- Arduino with attached shield
- 1 × Touch sensor
- 1 × Servo motor
- RMC wires
- USB cable

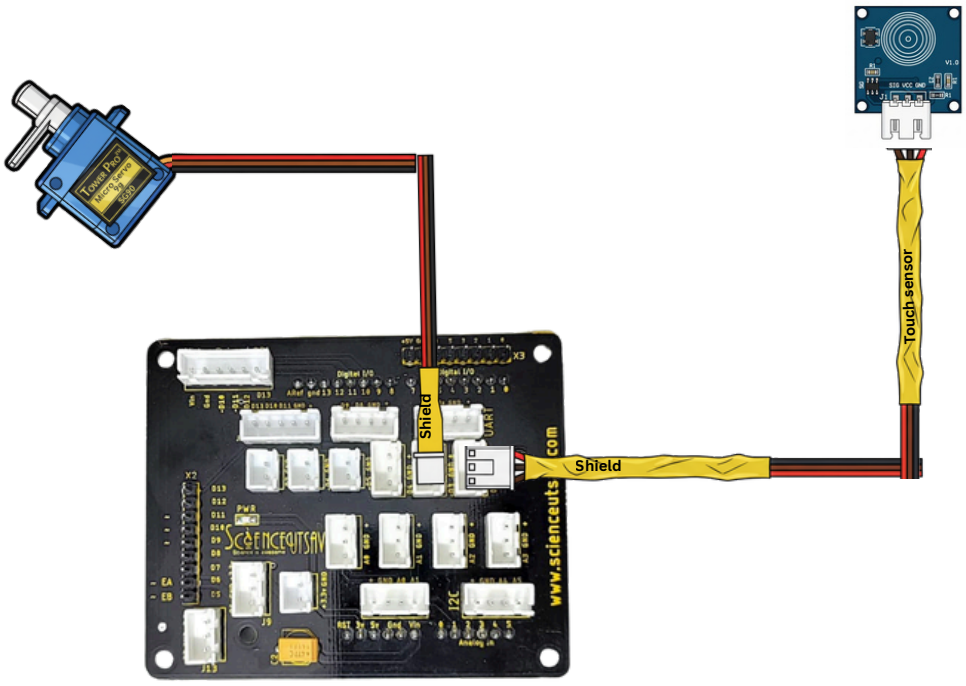
How to Connect

Use RMC wires to connect :

- 1 × Touch sensor → Arduino pin D2 and GND
- 1 × Servo motor → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <Servo.h>

#define TOUCH_PIN 2   // Touch sensor on D2
#define SERVO_PIN 3   // Servo on D3

Servo myServo;

void setup() {
  pinMode(TOUCH_PIN, INPUT);
  myServo.attach(SERVO_PIN);
  myServo.write(0); // Start at 0°
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  if (touchState == HIGH) {  // Touch detected
    // Move from 0 → 180
    for (int angle = 0; angle <= 180; angle++) {
      myServo.write(angle);
      delay(10);
    }
  } else {
    myServo.write(0); // Return to 0°
  }
}
```

Code Explanation

- In this Arduino program, a touch sensor is connected to pin 2 (TOUCH_PIN) and a servo motor is connected to pin 3 (SERVO_PIN). The Servo library is used to control the servo.
- The setup() function runs once when the Arduino starts. It sets the touch sensor as INPUT, attaches the servo to its pin using attach(), and sets its initial position to 0°.
- The loop() function runs continuously. It reads the touch sensor value using digitalRead() and stores it in touchState.
- If the sensor is touched (HIGH), a for loop moves the servo gradually from 0° to 180° using write(), creating smooth motion.
- If the sensor is not touched (LOW), the servo returns to 0°.

So overall, the servo moves when the touch sensor is activated and returns back when it is not.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Servo Angle Control but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <Servo.h>

#define TOUCH_PIN 2    // Touch sensor on D2
#define SERVO_PIN 3    // Servo on D3
#define BUZZER_PIN 5   // Buzzer on D5

Servo myServo;

void setup() {
  pinMode(TOUCH_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  myServo.attach(SERVO_PIN);
  myServo.write(0); // Start at 0°

  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  int touchState = digitalRead(TOUCH_PIN);

  if (touchState == HIGH) {  // Touch detected
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON

    // Move from 0 → 180
    for (int angle = 0; angle <= 180; angle++) {
      myServo.write(angle);
      delay(10);
    }

  } else {
    digitalWrite(BUZZER_PIN, LOW); // Buzzer OFF
    myServo.write(0);              // Return to 0°
  }
}
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Servo Angle Control how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Precision Servo Logic

Class Explanation

A Precision Servo Logic system combines servo motor control with logical input processing to achieve accurate and reliable movement. It uses control signals and logic conditions to position the servo precisely, enabling advanced automation, robotics control, and intelligent motion-based applications.

Materials Required

- Arduino with attached shield
- 1 × IR sensor
- 1 × Servo motor
- RMC wires
- USB cable

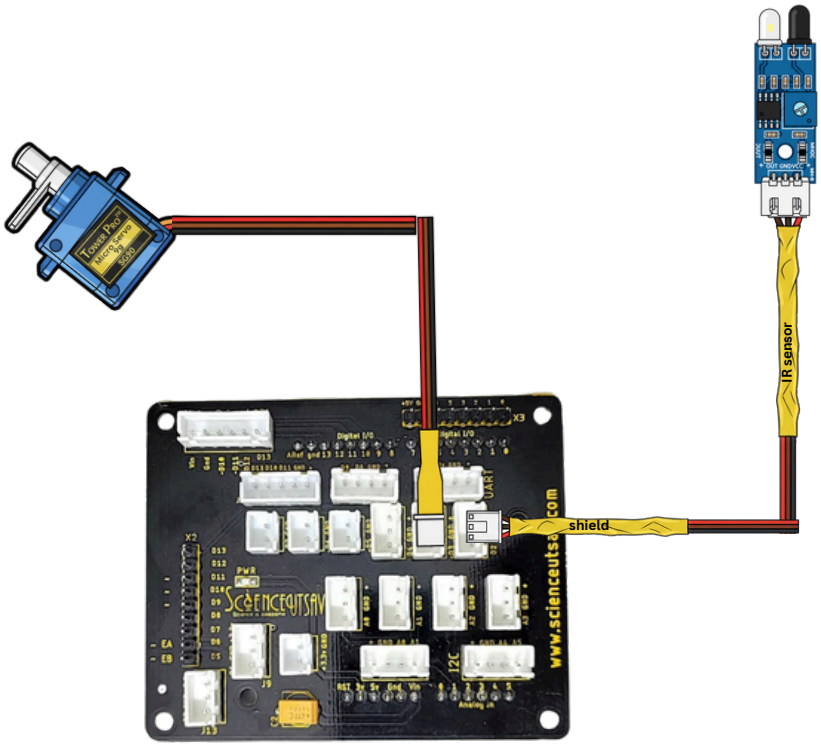
How to Connect

Use RMC wires to connect :

- 1 × IR sensor → Arduino pin D2 and GND
- 1 × Servo motor → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <Servo.h>

#define IR_PIN 2      // IR sensor on D2
#define SERVO_PIN 3   // Servo on D3

Servo myServo;

void setup() {
  pinMode(IR_PIN, INPUT);
  myServo.attach(SERVO_PIN);
  myServo.write(0); // Start at 0°
}

void loop() {
  int irState = digitalRead(IR_PIN);

  if (irState == LOW) {  // Object detected
    // Move from 0 → 180
    for (int angle = 0; angle <= 180; angle++) {
      myServo.write(angle);
      delay(10);
    }
  } else {
    // Return to 0
    myServo.write(0);
  }
}
```

Code Explanation

- In this Arduino program, an IR sensor is connected to pin 2 (IR_PIN) and a servo motor is connected to pin 3 (SERVO_PIN). The Servo library is used to control the servo.
- The setup() function runs once when the Arduino starts. It sets the IR sensor as INPUT, attaches the servo to its pin using attach(), and sets its initial position to 0°.
- The loop() function runs continuously. It reads the IR sensor value using digitalRead() and stores it in irState.
- If an object is detected (LOW signal), a for loop moves the servo gradually from 0° to 180° using write(), creating smooth motion.
- If no object is detected (HIGH), the servo returns to 0° position.

So overall, the servo moves when an object is detected by the IR sensor and returns back when there is no object.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Precision Servo Logic but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <Servo.h>

#define IR_PIN 2      // IR sensor on D2
#define SERVO_PIN 3   // Servo on D3
#define LED_PIN 5     // LED on D5

Servo myServo;

void setup() {
  pinMode(IR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  myServo.attach(SERVO_PIN);
  myServo.write(0); // Start at 0°

  digitalWrite(LED_PIN, LOW);
}

void loop() {
  int irState = digitalRead(IR_PIN);

  if (irState == LOW) {  // Object detected
    digitalWrite(LED_PIN, HIGH); // LED ON

    // Move servo 0 → 180
    for (int angle = 0; angle <= 180; angle++) {
      myServo.write(angle);
      delay(10);
    }
  } else {
    digitalWrite(LED_PIN, LOW); // LED OFF
    myServo.write(0);           // Return to 0°
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:
the basic idea of Precision Servo Logic how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how servo motors produce controlled movement and how signals determine their position and motion.
- Learn the basic working of a servo motor and how to control simple movements and positioning.
- Understand how PWM signals control the exact angle of a servo motor for precise positioning.
- Learn how logic control is combined with servo operation to achieve accurate and intelligent motion in electronic systems.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output